

**CENTRO UNIVERSITÁRIO DINÂMICA DAS CATARATAS
CURSO DE SISTEMAS DE INFORMAÇÃO**

ILAN WENDLING THOELE

**COMUNICAÇÃO SÍNCRONA VIA REST E ASSÍNCRONA
ORIENTADA A EVENTOS: UMA ANÁLISE
EXPERIMENTAL NO PROCESSAMENTO DE REGISTROS**

FOZ DO IGUAÇU

2026

ILAN WENDLING THOELE

COMUNICAÇÃO SÍNCRONA VIA REST E ASSÍNCRONA
ORIENTADA A EVENTOS: UMA ANÁLISE EXPERIMENTAL NO
PROCESSAMENTO DE REGISTROS

Trabalho de Conclusão de Curso apresentado como
requisito obrigatório para obtenção do título de Ba-
charel em Sistemas de Informação do Centro Univer-
sitário Dinâmica das Cataratas.

Orientador: Professora Alessandra Bussador

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA

Em atendimento ao disposto no Art. 9º, inciso I, alíneas “c”, “d” e “f” da Portaria CNPq nº 2.664, de 6 de março de 2026, que institui a Política de Integridade na Atividade Científica do CNPq, declara-se o uso de ferramentas de Inteligência Artificial Generativa (IAG) durante a elaboração do presente Trabalho de Conclusão de Curso.

Ferramentas utilizadas: Claude (Anthropic) e Codex (OpenAI), modelos de linguagem de propósito geral.

Finalidade do uso: apoio à reorganização do tema, estruturação textual, revisão gramatical e estilística, levantamento inicial de fontes, formatação acadêmica e apoio à elaboração do protocolo experimental.

Não foram utilizadas ferramentas de IAG para fabricar resultados experimentais, alterar medições, criar referências inexistentes ou substituir a análise crítica do autor. Todas as fontes citadas devem ser verificadas diretamente e os experimentos serão executados e interpretados pelo autor.

Em conformidade com a política institucional de integridade científica, o autor assume integral responsabilidade pelo conteúdo final, pelas decisões metodológicas, pelo código produzido, pelos dados coletados e pelas conclusões apresentadas.

RESUMO

Este trabalho propõe comparar experimentalmente duas formas de comunicação aplicadas ao processamento de registros de auditoria: uma implementação síncrona baseada em API REST e uma implementação assíncrona orientada a eventos com Apache Kafka. As variantes compartilham o mesmo contrato de dados, a mesma regra de validação e a mesma persistência em PostgreSQL. A comparação futura considerará métricas de desempenho e cenários de indisponibilidade e recuperação, sem pressupor superioridade de uma abordagem. Esta versão documenta a fundamentação, o método planejado e o estado atual do protótipo, sem apresentar resultados experimentais.

Palavras-chaves: Arquitetura orientada a eventos. REST. Registros de auditoria. Apache Kafka.

LISTA DE FIGURAS

FIGURA 1 – Visão geral do sistema experimental	25
FIGURA 2 – Diagrama inicial de casos de uso	26
FIGURA 3 – Atividade de processamento síncrono	27
FIGURA 4 – Atividade de processamento assíncrono	28
FIGURA 5 – Diagrama preliminar de classes	28
FIGURA 6 – Arquitetura da variante síncrona	29
FIGURA 7 – Arquitetura da variante assíncrona	29
FIGURA 8 – Sequência da variante síncrona	29
FIGURA 9 – Sequência da variante assíncrona	30
FIGURA 10 – Diagrama de componentes existentes	30
FIGURA 11 – Modelo de dados simplificado	30

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Problema de pesquisa	8
1.2	Justificativa	8
1.3	Objetivos	8
1.3.1	Objetivo geral	8
1.3.2	Objetivos específicos	9
1.4	Hipóteses de trabalho	9
1.5	Delimitação	9
1.6	Organização do trabalho	9
2	REVISÃO BIBLIOGRÁFICA	10
2.1	Registros e trilhas de auditoria	10
2.2	Comunicação em sistemas distribuídos	11
2.2.1	HTTP, REST e comunicação síncrona	11
2.2.2	Mensageria e comunicação assíncrona	12
2.3	Arquitetura orientada a eventos	13
2.4	Apache Kafka	14
2.5	Desempenho, resiliência e observabilidade	14
2.6	Trabalhos relacionados	15
2.7	Síntese	17
3	METODOLOGIA	18
3.1	Tipo de pesquisa	18
3.1.1	Natureza da pesquisa	18
3.1.2	Abordagem metodológica	18
3.1.3	Procedimentos técnicos	18
3.2	Procedimentos metodológicos	19
3.2.1	Métodos de coleta de informações	19
3.2.2	Métodos de análise das informações	19
3.3	Metodologia de desenvolvimento do sistema	20
3.3.1	Modelo de desenvolvimento de software	20
3.3.2	Tecnologias utilizadas	20
3.3.3	Arquitetura do sistema	20
3.4	Procedimentos de teste e validação	21
3.4.1	Tipos de testes aplicados	21

3.4.2	Cenários de teste	21
3.4.3	Métricas coletadas	21
3.4.4	Critérios de aceitação	22
3.4.5	Protocolo de execução e reprodutibilidade	22
3.4.6	Ameaças à validade	23
4	DESENVOLVIMENTO DO SISTEMA	24
4.1	Visão geral do sistema	24
4.2	Levantamento e especificação de requisitos	24
4.2.1	Requisitos funcionais	24
4.2.2	Requisitos não funcionais	24
4.2.3	Regras de negócio	25
4.3	Modelagem UML	26
4.3.1	Diagrama de casos de uso	26
4.3.2	Diagrama de atividades	26
4.3.3	Diagrama de classes	26
4.3.4	Diagramas complementares	27
	REFERÊNCIAS	31

1 INTRODUÇÃO

Sistemas de informação registram continuamente ações executadas por usuários, serviços e componentes de infraestrutura. Autenticações, alterações cadastrais, concessões de permissão e mudanças de configuração são exemplos de ocorrências que podem compor uma trilha de auditoria. Segundo Kent e Souppaya (2006), a gestão de logs abrange geração, transmissão, armazenamento, análise e descarte dos registros produzidos por sistemas computacionais. A utilidade desses dados depende de sua produção consistente, de sua integridade e da possibilidade de recuperação posterior.

Em aplicações distribuídas, o componente que produz uma ocorrência nem sempre é o mesmo que valida, enriquece e persiste o registro de auditoria. Consequentemente, é necessário escolher uma forma de comunicação entre esses participantes. Em uma interação síncrona, o produtor envia uma requisição e aguarda a conclusão do processamento. O HTTP estabelece uma semântica padronizada para requisições e respostas, constituindo a base de muitas APIs REST (FIELDING; NOTTINGHAM; RESCHKE, 2022). Essa alternativa oferece confirmação imediata, porém cria dependência temporal entre os participantes.

Na comunicação assíncrona, a aceitação e o processamento podem ocorrer em momentos distintos. Um produtor publica uma mensagem, um broker a retém e um consumidor realiza o processamento. Esse mecanismo reduz o acoplamento temporal e permite absorver rajadas de carga, mas introduz questões de reentrega, duplicidade, ordenação, observabilidade e consistência temporal (HOHPE; WOOLF, 2003; HELLAND, 2007). Em uma arquitetura orientada a eventos, os componentes produzem e reagem a representações de fatos ocorridos. A arquitetura não deve ser confundida com uma tecnologia específica: o Apache Kafka é uma plataforma concreta de fluxo de eventos que pode ser utilizada em uma solução orientada a eventos (Apache Software Foundation, 2026b).

Estudos empíricos indicam que os efeitos dessas decisões dependem do contexto. Cabane e Farias (2024) compararam uma aplicação monolítica e uma implementação orientada a eventos e observaram diferenças em tempo de resposta e consumo de recursos. Kazanavičius e Mažeika (2023) avaliaram REST, RabbitMQ, Kafka, gRPC e GraphQL e concluíram que a seleção deve considerar necessidades e critérios específicos. Esses trabalhos desaconselham a suposição de que uma alternativa seja universalmente superior.

Este estudo utiliza o processamento de registros de auditoria como cenário controlado. A funcionalidade será implementada em duas variantes com contrato, validação, cálculo de hash e persistência equivalentes. Na primeira, o cliente aguardará o processamento completo por uma API REST. Na segunda, a API produzirá um evento no Kafka e um consumidor o processará posteriormente. Essa equivalência funcional permitirá observar trade-offs de desempenho e resiliência sem comparar regras de negócio distintas.

1.1 PROBLEMA DE PESQUISA

A adoção de comunicação assíncrona pode reduzir o tempo pelo qual o cliente permanece bloqueado e permitir retenção de mensagens durante indisponibilidades do consumidor. Em contrapartida, adiciona broker, consumidor, gerenciamento de offsets e necessidade de idempotência. A comunicação síncrona apresenta um fluxo mais direto, mas o produtor depende da disponibilidade e do tempo de processamento do serviço de auditoria.

Entre os trabalhos analisados, foram identificadas avaliações de arquiteturas monolíticas e distribuídas, de diferentes tecnologias de comunicação e do impacto de EDA sobre desempenho (BLINOWSKI; OJDOWSKA; PRZYBYŁEK, 2022; CABANE; FARIAS, 2024; KAZANAVIČIUS; MAŽEIKAS, 2023). O recorte deste trabalho não parte da afirmação de inexistência de estudos. Busca-se produzir evidência experimental adicional em um cenário delimitado, no qual as duas variantes executam a mesma função de auditoria e são avaliadas também durante indisponibilidade e recuperação.

Formula-se, portanto, a seguinte pergunta de pesquisa:

Quais trade-offs de desempenho e resiliência são observados entre a comunicação síncrona via REST e a comunicação assíncrona orientada a eventos no processamento de registros de auditoria?

1.2 JUSTIFICATIVA

Do ponto de vista científico, a comparação controlada permite relacionar mecanismos de comunicação a métricas observáveis, evitando recomendações baseadas somente em características teóricas. Cabane e Farias (2024) destacam a necessidade de ampliar a evidência empírica sobre o impacto da arquitetura orientada a eventos. O estudo também explicitará seu protocolo, variáveis, repetições e ameaças à validade, conforme as orientações para experimentação em engenharia de software de Wohlin et al. (2012).

Do ponto de vista prático, arquitetos precisam decidir se o desacoplamento oferecido por mensageria compensa sua complexidade operacional. Registros de auditoria podem ser produzidos em rajadas e precisam permanecer disponíveis para investigação, mas normalmente não integram o resultado funcional que deve ser exibido imediatamente ao usuário. Perda, duplicidade ou atraso excessivo, contudo, reduzem o valor da trilha de auditoria (KENT; SOUPPAYA, 2006; OWASP Foundation, 2026).

1.3 OBJETIVOS

1.3.1 Objetivo geral

Comparar experimentalmente o desempenho e a resiliência de uma implementação síncrona baseada em REST e de uma implementação assíncrona orientada a eventos no processamento de registros de auditoria.

1.3.2 Objetivos específicos

- a) definir um modelo comum de registro de auditoria e uma regra de processamento equivalente;
- b) implementar uma variante síncrona baseada em API REST e PostgreSQL;
- c) implementar uma variante assíncrona baseada em Apache Kafka, consumidor e PostgreSQL;
- d) definir cenários controlados de carga, indisponibilidade e recuperação;
- e) coletar posteriormente latência, throughput, taxa de erro, uso de recursos, perda, duplicidade e tempo de recuperação;
- f) analisar os trade-offs observados sem pressupor superioridade de uma alternativa.

1.4 HIPÓTESES DE TRABALHO

As hipóteses orientam o experimento futuro e não representam resultados já obtidos. Espera-se que a variante síncrona apresente menor complexidade e menor custo básico em cargas reduzidas. Espera-se também que a variante assíncrona reduza a latência de aceitação e retenha eventos durante a indisponibilidade do consumidor, ao custo de maior uso de recursos e diferença entre aceitação e conclusão. Todas as hipóteses poderão ser refutadas pelos dados.

1.5 DELIMITAÇÃO

O estudo limita-se a REST e Kafka. Não serão comparados RabbitMQ, gRPC ou GraphQL. Não fazem parte do escopo Kubernetes, Debezium, Event Sourcing, CQRS, Saga, replicação geográfica ou integrações externas. O ambiente será local e containerizado. Os eventos utilizarão dados sintéticos e não conterão credenciais ou dados pessoais reais. As conclusões futuras serão restritas às versões, configurações, cargas e recursos documentados.

1.6 ORGANIZAÇÃO DO TRABALHO

O Capítulo 2 apresenta a revisão bibliográfica. O Capítulo 3 descreve os materiais e o método experimental. O Capítulo 4 documenta o estado atual do protótipo e seus diagramas. Os capítulos posteriores apresentarão os resultados obtidos, sua discussão e as conclusões da pesquisa após a execução dos experimentos.

2 REVISÃO BIBLIOGRÁFICA

Este capítulo fundamenta as decisões que serão comparadas no experimento. A exposição distingue quatro níveis frequentemente tratados como equivalentes: HTTP é um protocolo de aplicação; REST é um estilo arquitetural; arquitetura orientada a eventos (EDA) organiza a colaboração entre componentes a partir da produção e reação a eventos; e Apache Kafka é uma plataforma que pode ser utilizada para implementar fluxos assíncronos. A distinção é necessária porque os efeitos observados não decorrem apenas da tecnologia, mas também das garantias adotadas pela aplicação, do momento em que uma operação é considerada concluída e do modo como falhas e repetições são tratadas.

2.1 REGISTROS E TRILHAS DE AUDITORIA

Registros de auditoria descrevem ocorrências atribuíveis em um sistema e apoiam reconstrução cronológica, responsabilização e investigação posterior. Embora possam compartilhar infraestrutura com logs operacionais e de depuração, sua finalidade é diferente: um log de depuração auxilia o desenvolvimento, enquanto uma trilha de auditoria deve permitir relacionar uma ação a um instante, uma origem, uma entidade e, quando aplicável, um ator. O NIST destaca que a gestão de logs envolve geração, transmissão, armazenamento, análise e descarte, e não somente a escrita de mensagens em arquivo (KENT; SOUPPAYA, 2006). A OWASP recomenda registrar informações suficientes para responder quando, onde, quem e o que ocorreu, sem incluir senhas, tokens ou outros segredos (OWASP Foundation, 2026).

No protótipo, cada evento contém UUID, tipo do evento, tipo e identificador da entidade, ator, origem, instante e conteúdo contextual. O UUID permite correlacionar emissão, aceitação, consumo e persistência. Essa correlação também viabiliza verificar perda e duplicidade sem depender apenas dos totais agregados. O conteúdo é serializado de maneira determinística antes do cálculo de SHA-256. A representação canônica evita que diferenças irrelevantes, como a ordem de propriedades de um objeto JSON, alterem o hash. O mecanismo permite detectar que o mesmo UUID foi associado a conteúdos divergentes; contudo, não constitui prova completa de não repúdio, pois essa propriedade também exigiria proteção de chaves, relógio confiável, controle de acesso e armazenamento resistente a adulteração.

Registros de auditoria são adequados ao experimento porque apresentam uma função pequena e repetível: receber, validar, identificar e persistir um fato. Essa simplicidade permite manter equivalência funcional entre as variantes. Em uso real, entretanto, a escolha arquitetural afeta o significado da confirmação. Uma resposta síncrona pode informar ao sistema de origem que o registro já está persistido. Em uma alternativa assíncrona, a aceitação pode ocorrer antes da persistência, exigindo mecanismos adicionais

para acompanhar backlog, falhas e conclusão posterior.

2.2 COMUNICAÇÃO EM SISTEMAS DISTRIBUÍDOS

Uma chamada entre processos não possui as mesmas garantias de uma chamada local. Rede, processo remoto e armazenamento podem falhar de modo independente, produzindo falhas parciais. O cliente pode receber timeout sem saber se a operação não iniciou, está em execução ou foi concluída antes de a resposta se perder. Por essa razão, repetição, timeout, identificação e observabilidade integram a semântica prática de uma operação distribuída.

Comunicação síncrona e assíncrona alteram onde essa incerteza aparece. Na interação síncrona, o cliente permanece temporalmente acoplado ao serviço e normalmente interpreta a resposta como conclusão do trabalho acordado. Na interação assíncrona, produtor e consumidor não precisam estar ativos no mesmo instante, mas a aplicação passa a distinguir aceitação, armazenamento intermediário e conclusão. O desacoplamento temporal reduz dependência simultânea, porém introduz estados intermediários que precisam ser operados e observados (HOHPE; WOOLF, 2003; HELLAND, 2007).

2.2.1 HTTP, REST e comunicação síncrona

HTTP estabelece semântica de requisições, respostas, métodos e códigos de estado (FIELDING; NOTTINGHAM; RESCHKE, 2022). REST, definido por Fielding (2000), acrescenta restrições arquiteturais como cliente-servidor, ausência de estado de sessão no servidor, possibilidade de cache, interface uniforme e sistema em camadas. Portanto, uma API que utiliza HTTP e JSON não é automaticamente uma implementação completa de REST. Neste estudo, a expressão “variante REST” identifica uma API orientada ao recurso de registro, utilizando HTTP de forma síncrona; a comparação não pretende avaliar individualmente todas as restrições do estilo.

Na restrição *stateless*, cada requisição deve conter as informações necessárias para ser compreendida, sem depender de contexto de sessão mantido entre requisições no servidor. Segundo Fielding (2000), essa restrição melhora visibilidade, pois cada interação pode ser analisada isoladamente; favorece recuperação de falhas parciais; e permite liberar recursos entre requisições, facilitando distribuição de carga entre réplicas. A ausência de estado conversacional não significa ausência de dados persistidos: o servidor pode e normalmente precisa consultar bancos, mas não deve depender de uma sessão implícita para interpretar a requisição.

O mesmo autor apresenta limitações. A repetição de contexto em cada mensagem aumenta o volume transferido e o estado mantido no cliente reduz o controle do servidor sobre a sequência de interação. Além disso, a escalabilidade possibilitada por *statelessness* não elimina gargalos em processamento, banco ou dependências posteriores. Uma API sem

sessão ainda pode ficar indisponível se todas as requisições dependerem do mesmo banco de dados.

No fluxo estudado, o cliente envia `POST /audit`; a API valida o contrato, calcula o hash, persiste e somente então responde `201 Created`. Essa definição oferece uma pós-condição direta: ao receber sucesso, o cliente sabe que existe um registro persistido. Em contrapartida, validação, processamento e banco integram o caminho crítico. A latência do banco é incorporada à latência observada pelo cliente, conexões permanecem ocupadas e a indisponibilidade de uma dependência impede a conclusão. Timeouts e novas tentativas também exigem idempotência, pois a ausência de resposta não demonstra que o efeito não ocorreu.

Na operação diária, REST oferece ferramentas amplamente disponíveis, inspeção simples das mensagens e uma relação direta entre requisição e resposta. Balanceamento horizontal é facilitado quando não há sessão local, mas capacidade adicional da API não resolve automaticamente saturação do banco. Monitoramento costuma acompanhar taxa de requisições, duração, códigos HTTP, timeouts e saturação de pools. O custo operacional inicial tende a ser menor do que em uma topologia que inclui broker e consumidor, embora políticas de retry, circuit breaker e limitação de carga ainda possam ser necessárias.

2.2.2 Mensageria e comunicação assíncrona

Mensageria introduz produtor, intermediário e consumidor. O produtor registra uma mensagem no broker sem executar diretamente a lógica do consumidor. A retenção permite que produtor e consumidor operem em ritmos distintos e que o processamento seja retomado após indisponibilidade. Esse mecanismo pode absorver rajadas, mas não elimina o trabalho: quando a taxa de entrada supera a taxa de processamento, forma-se backlog, aumentando a latência de conclusão e a necessidade de capacidade posterior.

As expressões *at-most-once*, *at-least-once* e *exactly-once* precisam ser associadas ao limite em que são garantidas. A idempotência do produtor Kafka evita duplicações causadas por certas repetições de envio ao broker, mas não torna automaticamente idempotente o efeito de um consumidor em um banco externo (Apache Software Foundation, 2026a). Se o consumidor persistir no PostgreSQL e falhar antes de confirmar o offset, a mensagem poderá ser entregue novamente. O protótipo adota, portanto, entrega pelo menos uma vez e efeito idempotente: a restrição única sobre `event_id` impede uma segunda linha, e o hash diferencia reentrega legítima de conteúdo conflitante.

Idempotência possui desafios práticos. O identificador precisa ser estável desde a origem; a verificação e o efeito devem estar protegidos contra concorrência; a política para o mesmo identificador com conteúdo diferente deve ser explícita; e a retenção da informação de deduplicação precisa cobrir o período de reentrega. Apenas consultar e depois inserir não é suficiente sob concorrência, pois dois consumidores podem observar ausência simultaneamente. Uma restrição única no banco torna a decisão atômica no

ponto de persistência.

O uso de mensageria também muda manutenção e operação. O contrato do evento passa a ser uma dependência semântica entre produtor e consumidores. Alterações incompatíveis podem permanecer no tópico e falhar somente quando consumidas. Backlog, idade da mensagem, reentregas, registros não processáveis, estado dos grupos e diferença entre offsets produzidos e consumidos tornam-se sinais operacionais relevantes. Em compensação, a retenção permite recuperar consumidores e adicionar novos leitores sem bloquear o produtor.

2.3 ARQUITETURA ORIENTADA A EVENTOS

Um evento representa um fato relevante que já ocorreu, enquanto um comando expressa uma solicitação para que algo seja feito. Em EDA, componentes publicam eventos e outros componentes reagem a eles. A interação pode ocorrer por notificação simples, transferência do estado necessário ao consumidor ou fluxos de processamento de eventos. Kafka não define sozinho essa arquitetura: o sistema é orientado a eventos quando contratos, responsabilidades, tratamento de falhas e evolução são organizados em torno desses fatos.

Entre as vantagens potenciais estão desacoplamento temporal, extensão por novos consumidores e isolamento de rajadas. Um produtor pode continuar aceitando eventos enquanto um consumidor está temporariamente indisponível, desde que o broker permaneça disponível e haja capacidade de retenção. Novos consumidores podem reutilizar o fluxo sem modificar a API produtora. Esses benefícios são condicionais: contratos continuam acoplando semanticamente os participantes, e o atraso entre produção e consumo cria consistência eventual.

As limitações aparecem principalmente na compreensão do estado global. Uma resposta de aceitação não representa a conclusão do processo; falhas podem ser descobertas depois que o cliente encerrou a interação; e a ordem entre eventos depende da estratégia de particionamento. Laigner et al. (2021) observaram que consistência, evolução de esquemas e interleaving de fluxos transferem responsabilidades para a camada de aplicação. O diagnóstico requer identificadores de correlação e combinação de logs, métricas e rastreamento. O OpenTelemetry descreve traces como o caminho de uma operação através de processos e redes e recomenda correlação entre sinais de telemetria (OpenTelemetry Authors, 2026).

Em termos práticos, EDA pode ser apropriada para integração desacoplada, propagação de mudanças, telemetria e trabalhos que toleram conclusão posterior. Ela é menos adequada quando o usuário precisa de confirmação imediata do efeito ou quando a complexidade operacional não se justifica. Custos incluem infraestrutura do broker, armazenamento e tráfego, instrumentação, operação de consumidores, gestão de contratos e capacitação da equipe. Escalar consumidores aumenta capacidade apenas até limites

impostos por partições, banco de dados e trabalho por mensagem.

2.4 APACHE KAFKA

Kafka é uma plataforma de *event streaming* baseada em log distribuído. Produtores escrevem registros em tópicos; tópicos são divididos em partições; consumidores leem posições identificadas por offsets; e consumidores de um grupo dividem as partições. A retenção é independente do consumo, permitindo releitura. Brokers replicam partições para tolerar falhas conforme a configuração do cluster (Apache Software Foundation, 2026b; Apache Software Foundation, 2026a).

A ordenação é garantida dentro de uma partição, não de todo o tópico. Usar a mesma chave direciona eventos relacionados para a mesma partição quando o particionamento permanece compatível, preservando a ordem relativa desses eventos. Entretanto, aumentar o número de partições, alterar chaves ou combinar eventos de entidades distintas pode mudar a ordem global percebida. Há também um compromisso entre ordem e paralelismo: uma única partição oferece ordem total no tópico, mas limita o número de consumidores ativos no mesmo grupo.

Offsets representam a posição de consumo, e sua confirmação não é equivalente à persistência do efeito em um banco externo. Confirmar antes do efeito cria risco de perda lógica; confirmar depois permite reentrega. Uma transação que abranja Kafka e PostgreSQL não é fornecida automaticamente. Neste estudo, o consumidor persiste primeiro e confirma depois. A unicidade por UUID torna a reexecução segura quanto à duplicidade de linhas, embora continue necessário contabilizar reentregas e distinguir conflitos.

Outro ponto é a pressão de retorno. Kafka permite acumular mensagens quando consumidores ficam lentos, mas o backlog não pode crescer indefinidamente. Retenção, capacidade de disco, tempo máximo aceitável até a conclusão e velocidade de drenagem precisam ser monitorados. No experimento, a quantidade de mensagens pendentes e o tempo de recuperação após a retomada do consumidor representarão esse comportamento.

2.5 DESEMPENHO, RESILIÊNCIA E OBSERVABILIDADE

Latência é um intervalo entre marcos definidos, não uma propriedade única. Na variante síncrona, a latência será medida do início do envio até o recebimento da resposta emitida após a persistência. Na assíncrona, serão registrados dois intervalos: aceitação, do envio ao 202 **Accepted**; e conclusão, do envio até a confirmação da linha no PostgreSQL. Comparar o 202 diretamente com o 201 responderia apenas qual API devolve controle mais cedo, não qual fluxo conclui primeiro.

Média pode ocultar caudas. Por isso, serão apresentados mediana e percentis p95 e p99, que indicam limites abaixo dos quais se encontram, respectivamente, 95% e 99% das observações. Esses percentis são sensíveis a aquecimento, pausas do ambiente,

saturação de conexões, filas internas, coleta de lixo e contenção no banco. Serão calculados sobre amostras válidas de cada repetição, mantendo resultados individuais para mostrar dispersão.

Throughput é a quantidade de registros efetivamente concluídos por unidade de tempo. Na comparação principal, conclusão será persistência, evitando contar apenas respostas 202. A taxa oferecida pelo gerador será registrada separadamente da taxa concluída. Se a entrada superar a capacidade, o throughput pode permanecer estável enquanto backlog e latência aumentam; por isso, as três medidas precisam ser interpretadas em conjunto.

Taxa de erro será calculada como operações sem o resultado esperado divididas pelas operações tentadas. Erros HTTP, falhas de publicação, falhas de consumo e conflitos de integridade serão separados, pois possuem significados distintos. CPU e memória ajudam a interpretar eficiência, mas dependem do limite de recursos e da instrumentação. Na alternativa assíncrona, backlog, reentregas e tempo de drenagem completam a análise.

Neste estudo, resiliência indica a capacidade do sistema de manter sua função ou retomá-la após uma falha prevista. Durante a indisponibilidade do consumidor, a API pode continuar aceitando mensagens, mas isso não significa que o fluxo completo continue concluindo registros. Serão observados disponibilidade da entrada, quantidade aceita, quantidade persistida, perda, duplicidade, backlog máximo e tempo necessário para retornar ao estado estável.

Observabilidade afeta a própria capacidade de validar o experimento. Cada evento terá UUID propagado nos logs e armazenado no banco. Logs registrarão marcos de recebimento, publicação, consumo, tentativa de persistência e confirmação. Essa correlação permite calcular a latência de conclusão e investigar valores extremos. Em sistemas reais, traces distribuídos, métricas e logs correlacionados reduzem o esforço de diagnóstico, mas também acrescentam volume de telemetria e custo operacional (OpenTelemetry Authors, 2026).

2.6 TRABALHOS RELACIONADOS

Cabane e Farias (2024) compararam duas versões funcionalmente relacionadas da mesma aplicação, uma monolítica e outra orientada a eventos. Foram avaliados CPU, memória, tempo de resposta, throughput e tráfego em cenários realistas. Os resultados indicaram menor consumo de recursos e melhores tempos de resposta para a versão monolítica no contexto estudado. O próprio estudo limita a generalização às configurações e aplicação avaliadas. Sua contribuição para este TCC é metodológica: reforça equivalência funcional, múltiplas métricas e análise estatística. A diferença é que Cabane e Farias comparam estilos mais amplos, enquanto este trabalho isola comunicação síncrona REST e comunicação assíncrona Kafka, além de separar aceitação e conclusão e introduzir falhas controladas.

Kazanavičius e Mažeika (2023) avaliaram REST, RabbitMQ, Kafka, gRPC e GraphQL durante decomposição de monólitos. A amplitude permite discutir critérios de seleção, mas também distribui a análise entre cinco mecanismos e um contexto de migração. O estudo sustenta que a escolha depende dos requisitos e não apenas de desempenho bruto. O presente trabalho reduz o número de alternativas para aprofundar os efeitos temporais e de falha de duas variantes equivalentes, sem avaliar decomposição organizacional ou migração de um monólito.

Blinowski, Ojdowska e Przybyłek (2022) implementaram quatro versões de uma aplicação de referência, combinando arquitetura monolítica ou de microsserviços e tecnologias distintas. Os autores mostram que custos de distribuição e resultados de escalabilidade dependem da implementação e do ambiente. Essa combinação também constitui uma limitação para o recorte atual: estilo, tecnologia e topologia podem variar simultaneamente. Neste TCC, linguagem, contrato, processamento e banco são mantidos, reduzindo a quantidade de fatores alterados.

Aksakalli et al. (2021) revisaram 239 publicações e selecionaram 38 estudos primários, identificando três abordagens de implantação, sete padrões de comunicação, oito desafios de implantação e seis desafios de comunicação. A revisão demonstra que implantação e comunicação são decisões igualmente relevantes e que padrões diferentes atendem contextos distintos. Por ser um estudo secundário, não fornece uma medição direta do comportamento de REST e Kafka sob a mesma carga; ele apoia a seleção dos conceitos e dos desafios que precisam ser observados experimentalmente.

Laigner et al. (2021) combinaram literatura, inspeção de aplicações abertas e consulta a mais de 120 profissionais e pesquisadores para caracterizar gestão de dados em microsserviços. Foram relatados desafios de consistência eventual, evolução de esquemas, interleaving de eventos, replicação e validação de restrições na aplicação. O estudo explica por que idempotência e integridade não podem ser tratadas apenas como propriedades do broker. Entretanto, seu objetivo é caracterizar práticas de dados, e não comparar desempenho e recuperação de dois fluxos equivalentes.

Os estudos convergem em três aspectos: distribuição adiciona custos; resultados dependem do contexto; e garantias de dados e operação não surgem automaticamente da tecnologia. Diferem quanto ao objeto, às alternativas e às métricas. No conjunto analisado, a oportunidade específica deste trabalho é comparar uma confirmação síncrona após persistência com uma aceitação assíncrona seguida de conclusão posterior, usando o mesmo contrato e armazenamento, e observar não apenas carga normal, mas indisponibilidade e drenagem de backlog. Essa lacuna é apresentada como recorte da busca e dos estudos selecionados, não como afirmação de inexistência absoluta de trabalhos.

2.7 SÍNTESE

REST favorece uma interação direta, com semântica de conclusão mais simples, e sua ausência de estado conversacional pode facilitar replicação e recuperação de requisições. O caminho crítico, contudo, incorpora as dependências necessárias ao efeito. EDA e Kafka permitem desacoplar produção e consumo e absorver diferenças temporárias de ritmo, mas introduzem consistência eventual, backlog, reentrega, ordenação por partição e maior exigência de observabilidade. O experimento será estruturado para medir essas diferenças sem confundir aceitação com conclusão e sem atribuir ao estilo arquitetural efeitos causados por contratos ou regras distintas.

3 METODOLOGIA

Este capítulo descreve o planejamento da pesquisa, do desenvolvimento e da avaliação. Procedimentos ainda não executados são apresentados no futuro e não constituem resultados. O protocolo é especificado antes da coleta definitiva para reduzir decisões motivadas pelos dados observados e permitir repetição por terceiros.

3.1 TIPO DE PESQUISA

3.1.1 Natureza da pesquisa

A pesquisa é aplicada porque produz um protótipo e evidências destinadas a apoiar uma decisão concreta de engenharia de software. Também é experimental porque manipulará um fator — a forma de comunicação — e observará seus efeitos em condições controladas. Esse desenho é adequado à pergunta, pois desempenho e recuperação não podem ser concluídos apenas por argumentação teórica: precisam ser medidos em implementações executáveis (JAIN, 1991; WOHLIN et al., 2012).

3.1.2 Abordagem metodológica

A abordagem é quantitativa. Latência, throughput, erros, uso de recursos, backlog, perda, duplicidade e recuperação serão representados numericamente. A abordagem permite comparar distribuições e magnitudes, e não somente relatar percepção do desenvolvedor. Análise qualitativa será utilizada para explicar mecanismos, limitações e ameaças, mas não substituirá os dados.

A relação com os objetivos específicos é direta. A definição do contrato e das regras comuns atende ao objetivo de estabelecer equivalência funcional. A construção das variantes atende aos objetivos de implementar REST e Kafka. A instrumentação e os cenários operacionalizam o objetivo de coletar métricas. A análise estatística e a interpretação conjunta das métricas atendem ao objetivo de identificar trade-offs, sem reduzir a conclusão a uma classificação única.

3.1.3 Procedimentos técnicos

Os procedimentos incluem pesquisa bibliográfica narrativa orientada ao problema, desenvolvimento incremental, testes funcionais e experimento controlado. A pesquisa bibliográfica fundamenta conceitos e confronta trabalhos relacionados; não é o método principal de resposta. A pergunta será respondida pela combinação entre protótipo, protocolo reproduzível e observações quantitativas.

3.2 PROCEDIMENTOS METODOLÓGICOS

3.2.1 Métodos de coleta de informações

As fontes teóricas incluem artigos revisados por pares, livros acadêmicos, RFCs, normas e documentação oficial. Para os dados experimentais, cada evento possuirá UUID gerado antes do envio e um timestamp de origem. Esse identificador aparecerá no arquivo de carga, nos logs das APIs e do consumidor e na linha persistida, permitindo conciliar a trajetória sem inferência por posição ou horário aproximado.

Um script versionado preparará payloads sintéticos e executará a carga. A ferramenta prevista é k6, cuja versão será fixada no repositório e na imagem de execução antes do piloto. O script registrará identificador do ensaio, UUID, instante inicial, resposta HTTP, duração e erro. Na variante assíncrona, um script de conciliação consultará o PostgreSQL pelo UUID e calculará o intervalo até `occurred_at/persisted_at`, sem tratar o 202 como conclusão.

Logs estruturados em JSON Lines registrarão `run_id`, `event_id`, componente, etapa, instante e resultado. Métricas de CPU e memória serão coletadas por contêiner em intervalos regulares por meio de estatísticas do runtime Docker; versões e comando serão armazenados com o ensaio. Consultas SQL exportarão contagens e timestamps. Ao final, um script Python versionado produzirá CSVs normalizados a partir dos arquivos brutos. Essas ferramentas são planejadas e somente serão declaradas implementadas após sua incorporação e validação no repositório.

3.2.2 Métodos de análise das informações

A unidade de observação de latência será um evento concluído. A unidade de repetição será uma execução independente do cenário. Para cada repetição serão calculados quantidade oferecida, aceita e persistida; média, mediana, desvio e percentis p95 e p99; throughput concluído; erros por categoria; backlog máximo; reentregas; duplicidades persistidas; e tempo de recuperação.

Os resultados serão apresentados por cenário e repetição antes da agregação. A distribuição será inspecionada graficamente e por testes apropriados. Se os pressupostos paramétricos forem plausíveis, poderão ser utilizados intervalos e comparação de médias; caso contrário, serão preferidos métodos não paramétricos e intervalos por reamostragem. Serão relatados intervalo de confiança e tamanho de efeito quando aplicáveis. Significância estatística não será interpretada isoladamente: uma diferença pequena pode não justificar complexidade operacional, e uma arquitetura com menor latência de aceitação pode apresentar maior latência de conclusão.

3.3 METODOLOGIA DE DESENVOLVIMENTO DO SISTEMA

3.3.1 Modelo de desenvolvimento de software

Será adotado desenvolvimento incremental. O primeiro incremento define contrato, hash e persistência; o segundo implementa a API síncrona; o terceiro implementa produtor, Kafka e consumidor; o quarto acrescenta instrumentação e scripts de carga; o quinto valida o protocolo em piloto; e o sexto executa a coleta definitiva. Cada incremento deve manter código, configuração, testes e documentação consistentes.

3.3.2 Tecnologias utilizadas

O protótipo utiliza Python 3.12, FastAPI, PostgreSQL 16, Apache Kafka 3.9 e Docker Compose. FastAPI foi selecionado por oferecer validação declarativa e uma implementação pequena para as duas APIs. PostgreSQL oferece transação e restrição de unicidade, necessárias à idempotência. Kafka fornece retenção, partições, grupos e offsets. Docker Compose descreve a topologia local e reduz diferenças de instalação. k6 e scripts Python estão previstos para geração de carga e consolidação; suas versões serão fixadas antes da coleta.

Código, arquivo Compose, scripts, esquema, sementes e parâmetros serão versionados. Cada execução registrará hash do commit, sistema operacional, processador, memória, versões das imagens, limites de CPU/memória, número de partições, fator de replicação, configurações de produtor/consumidor, tamanho do pool do banco e propriedades da carga. Segredos e dados pessoais não integrarão os artefatos.

3.3.3 Arquitetura do sistema

As duas variantes compartilham `AuditEvent`, serialização canônica, SHA-256 e repositório PostgreSQL. Na síncrona, a API chama processamento e persistência antes de responder. Na assíncrona, a API valida e publica; o consumidor aplica posteriormente o mesmo processamento e persistência. Essa organização controla diferenças funcionais e permite atribuir o fator experimental à comunicação e ao momento de conclusão.

A confirmação de publicação no protótipo deverá ser validada no piloto. O 202 somente será emitido depois da condição de aceitação definida no protocolo; essa condição e a configuração de `acks` serão registradas. No consumidor, confirmação automática de offset permanecerá desabilitada, e o commit ocorrerá depois da persistência. O mesmo UUID com o mesmo hash será tratado como reentrega; o mesmo UUID com hash diferente será conflito de integridade.

3.4 PROCEDIMENTOS DE TESTE E VALIDAÇÃO

3.4.1 Tipos de testes aplicados

Testes unitários verificarão serialização determinística, SHA-256, validações e classificação de duplicidade/conflicto. Testes de integração executarão PostgreSQL e Kafka reais em contêineres, verificando criação do esquema, publicação, consumo, persistência e commit. Testes funcionais cobrirão códigos HTTP e pós-condições. Testes experimentais avaliarão desempenho e resiliência após a aprovação funcional.

Antes da medição, haverá um teste piloto. Seu objetivo não é produzir resultados do TCC, mas verificar scripts, relógios, correlação, limites e duração. Parâmetros poderão ser corrigidos durante o piloto; após o congelamento do protocolo, qualquer mudança relevante exigirá reinício da coleta das configurações afetadas.

3.4.2 Cenários de teste

No cenário C1, uma taxa baixa verificará correção sem saturação. No C2, níveis graduais avaliarão crescimento de latência e capacidade. No C3, uma rajada curta acima da taxa estável observará filas e absorção de pico. No C4, o consumidor da variante assíncrona e o processamento correspondente da variante síncrona ficarão temporariamente indisponíveis durante entrada ativa. No C5, o componente será retomado e serão medidos drenagem do backlog e tempo de recuperação.

O piloto estimará a maior taxa oferecida que mantém estabilidade por cinco minutos sem crescimento contínuo de backlog ou erros de saturação. Os níveis definitivos de C2 serão definidos como proporções dessa taxa de referência, inicialmente 25%, 50%, 75% e 100%. C1 utilizará o menor nível. C3 aplicará, por intervalo curto, taxa superior à referência definida pelo piloto. C4 manterá uma taxa estável escolhida no piloto e interromperá o componente por período fixo. C5 começa com a retomada e termina quando o backlog retornar ao nível anterior e permanecer estável.

Cada execução definitiva terá 60 segundos de aquecimento não incluídos na análise e cinco minutos de medição, salvo justificativa produzida pelo piloto. Serão realizadas no mínimo dez repetições independentes por combinação de variante e cenário. A ordem das variantes será alternada e randomizada dentro dos blocos de repetição. Entre execuções, banco, tópico, offsets e arquivos temporários serão reinicializados conforme procedimento automatizado, seguido de verificação de saúde.

3.4.3 Métricas coletadas

Latência síncrona será medida do início do envio até a resposta recebida após a persistência. Latência assíncrona de aceitação será medida do início do envio até o 202; latência assíncrona de conclusão, do mesmo início até a confirmação da linha no banco. Os relógios pertencem ao mesmo host experimental para reduzir erro de sincronização.

Timestamps serão monotônicos para duração quando a ferramenta permitir e UTC para correlação.

Throughput será a quantidade persistida dividida pela janela efetiva de medição. A taxa oferecida será reportada separadamente. Taxa de erro será calculada por etapa: HTTP, publicação, consumo, persistência e integridade. CPU e memória serão amostradas por contêiner. Backlog será estimado pela diferença de offsets e corroborado pela diferença entre aceitos e persistidos. Tempo de recuperação será o intervalo entre a retomada do componente e o retorno do backlog e throughput ao critério estável definido antes da coleta.

Perda será identificada quando um UUID emitido e aceito não possuir linha persistida após o encerramento e a janela de drenagem. Reentrega será contabilizada quando o consumidor observar novamente o mesmo UUID. Duplicidade persistida será contagem de UUIDs com mais de uma linha e deve permanecer zero pela restrição de unicidade. Conflito será o mesmo UUID associado a hash diferente.

3.4.4 Critérios de aceitação

O protótipo será funcional se: eventos válidos forem persistidos nas duas variantes; entradas inválidas forem rejeitadas; 201 somente ocorrer após persistência; 202 respeitar a condição de aceitação definida; o consumidor confirmar offset depois da persistência; reentrega idêntica não criar outra linha; conflito de hash produzir erro rastreável; e todos os UUIDs puderem ser conciliados.

O experimento será aceito se: versões e recursos estiverem registrados; scripts forem versionados; aquecimento for separado; número mínimo de repetições for concluído; ordem for alternada; dados brutos forem preservados; relógios e correlação forem validados; não houver intervenção manual não registrada; e a taxa realmente oferecida permanecer dentro da tolerância definida no piloto. Execuções com falha do próprio instrumento, reinicialização externa ou dados incompletos serão invalidadas mediante regra documentada, nunca apenas por apresentarem resultado desfavorável.

3.4.5 Protocolo de execução e reprodutibilidade

Para cada repetição, o executor deverá: registrar ambiente e commit; interromper a topologia anterior; remover apenas o estado experimental previsto; iniciar os contêineres; aguardar health checks; confirmar tópico e esquema; executar aquecimento; iniciar coleta de recursos e logs; aplicar carga; provocar a falha programada quando houver; aguardar a janela de drenagem; exportar banco e offsets; encerrar coletores; calcular checksums dos arquivos; e produzir manifesto do ensaio.

Os artefatos de uma execução serão organizados por `run_id`: manifesto de configuração, payloads, saída do gerador, logs JSONL, métricas de recursos, exportação de contagens e CSV normalizado. A análise será executada a partir desses arquivos, sem

consulta a dados modificáveis. O README informará comandos para construir, executar e analisar o ambiente.

3.4.6 Ameaças à validade

A validade interna pode ser afetada por processos concorrentes, aquecimento, caches, variação do runtime, coleta de telemetria e ordem dos ensaios. Serão utilizados host dedicado durante as execuções, aquecimento, repetições, alternância de ordem, limites iguais e registro de recursos. Instrumentação idêntica nas duas variantes reduzirá viés, embora seu overhead não seja nulo.

A validade de construto depende da definição correta das métricas. O risco principal é considerar 202 equivalente a persistência. A mitigação é separar aceitação e conclusão e usar UUID para confirmação. Throughput será definido por persistência, e disponibilidade da API não será confundida com disponibilidade do processo completo.

A validade de conclusão pode ser limitada por poucas repetições, autocorrelação e múltiplas comparações. Serão preservadas repetições independentes, distribuições e tamanhos de efeito, evitando conclusões baseadas somente em médias ou valores de p .

A validade externa é limitada pelo ambiente local, dados sintéticos, uma implementação, versões específicas e um domínio simplificado. Os resultados não serão generalizados para toda aplicação REST ou EDA. A mitigação consiste em documentar configurações e publicar artefatos reproduzíveis, permitindo replicações em outros ambientes. A validade de implementação será protegida por testes equivalentes e revisão do caminho executado, mas diferenças inerentes, como o broker adicional, permanecerão parte do fenômeno estudado.

4 DESENVOLVIMENTO DO SISTEMA

4.1 VISÃO GERAL DO SISTEMA

O sistema experimental processa registros de auditoria por duas formas de comunicação. Seu propósito é permitir uma comparação controlada, e não operar como plataforma corporativa de auditoria. O público-alvo é composto pelo pesquisador e por profissionais de engenharia de software interessados nos trade-offs entre comunicação síncrona e assíncrona.

As principais funções são receber um evento válido, calcular uma representação canônica e seu hash, persistir o registro e impedir efeitos duplicados. A variante REST conclui essas etapas antes da resposta. A variante orientada a eventos aceita a mensagem, publica no Kafka e delega a persistência a um consumidor. A Figura 1 apresenta as duas implementações submetidas ao mesmo processo experimental futuro.

No estado atual do protótipo estão implementados o contrato, o processamento comum, a persistência idempotente, as duas APIs, o produtor Kafka, o consumidor, a configuração Compose e testes unitários do hash. A execução integrada e os testes de carga permanecem como atividades planejadas e não são apresentadas como resultados concluídos.

4.2 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS

4.2.1 Requisitos funcionais

- RF01 receber registro de auditoria por endpoint HTTP;
- RF02 validar campos obrigatórios e limites do contrato;
- RF03 processar e persistir sincronamente um evento válido;
- RF04 publicar assincronamente um evento válido no Kafka;
- RF05 consumir eventos do tópico de auditoria;
- RF06 calcular hash SHA-256 da representação canônica;
- RF07 persistir o registro no PostgreSQL;
- RF08 identificar reentrega pelo UUID e impedir duplicidade persistida;
- RF09 confirmar o offset somente após persistência;
- RF10 expor verificação básica de saúde das APIs.

4.2.2 Requisitos não funcionais

- RNF01 utilizar o mesmo contrato e processamento nas duas variantes;

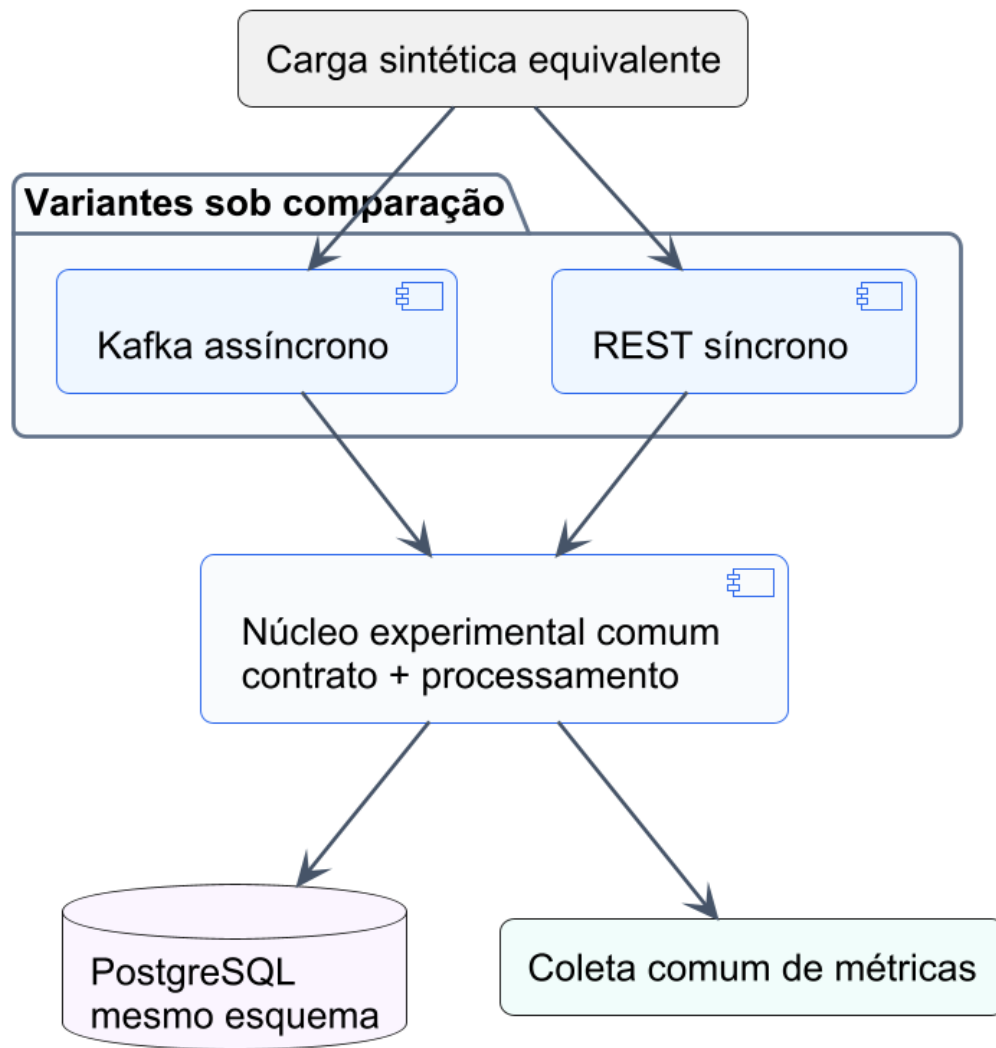


FIGURA 1 – Visão geral do sistema experimental

FONTE: Elaborado pelo autor (2026).

- RNF02 permitir execução local reproduzível por contêineres;
- RNF03 não registrar credenciais ou dados pessoais reais nos eventos sintéticos;
- RNF04 manter versões das dependências explicitadas;
- RNF05 preservar identificadores necessários à conciliação;
- RNF06 separar latência de aceitação e conclusão na variante assíncrona;
- RNF07 permitir reentrega sem repetir o efeito persistente;
- RNF08 produzir logs suficientes para rastrear processamento e falhas.

4.2.3 Regras de negócio

- RN01 todo evento deve possuir UUID único;

- RN02 todos os campos definidos no contrato são obrigatórios, exceto o payload, que pode ser vazio;
- RN03 o hash deve ser calculado sobre representação JSON canônica;
- RN04 o mesmo UUID e o mesmo hash caracterizam reentrega idempotente;
- RN05 o mesmo UUID com hash diferente caracteriza violação de integridade;
- RN06 a resposta 202 confirma aceitação, não conclusão do processamento;
- RN07 o consumidor somente confirma o offset após a persistência.

4.3 MODELAGEM UML

4.3.1 Diagrama de casos de uso

O ator primário é a aplicação cliente. Kafka e PostgreSQL aparecem como sistemas de suporte. O diagrama representa somente funcionalidades existentes ou diretamente previstas no escopo do protótipo.

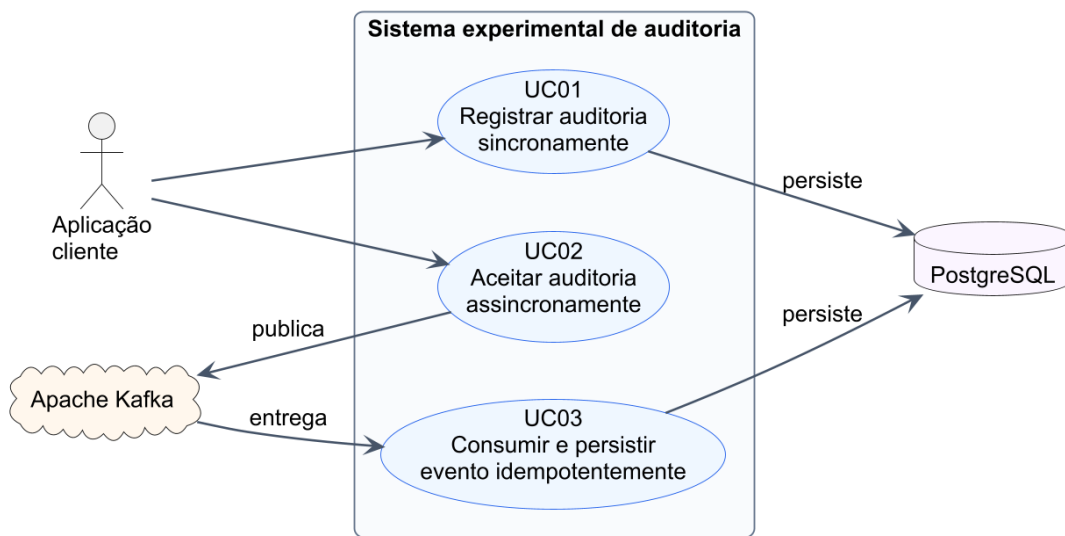


FIGURA 2 – Diagrama inicial de casos de uso

FONTE: Elaborado pelo autor (2026).

4.3.2 Diagrama de atividades

As Figuras 3 e 4 mostram os dois fluxos principais. No fluxo assíncrono, a separação entre aceitação e consumo é indicada por uma transição temporal descontínua.

4.3.3 Diagrama de classes

A versão preliminar do diagrama de classes deriva diretamente dos módulos Python. Funções de processamento e persistência são representadas como classes con-

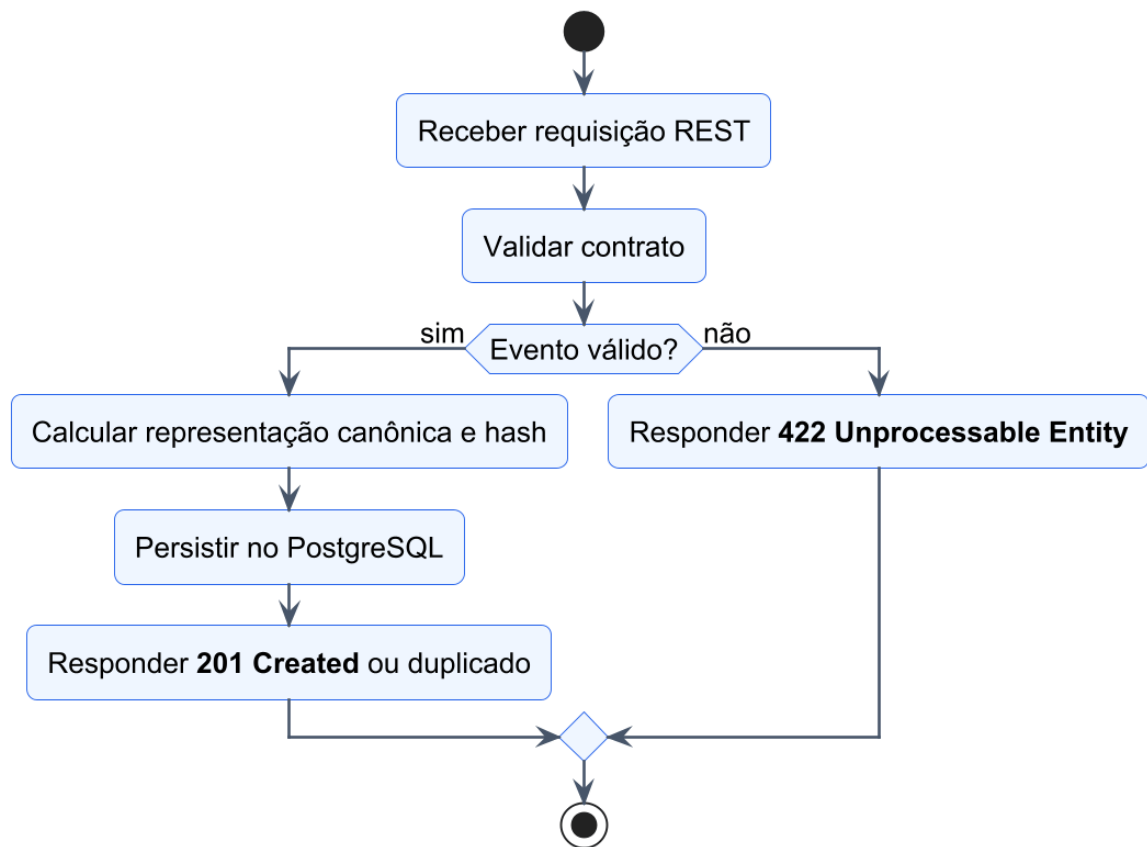


FIGURA 3 – Atividade de processamento síncrono

FONTE: Elaborado pelo autor (2026).

ceituais para comunicar responsabilidades, embora a implementação utilize funções de módulo em alguns casos.

4.3.4 Diagramas complementares

Os diagramas de arquitetura, sequência, componentes e dados complementam a UML obrigatória e compõem a via digital da implementação.

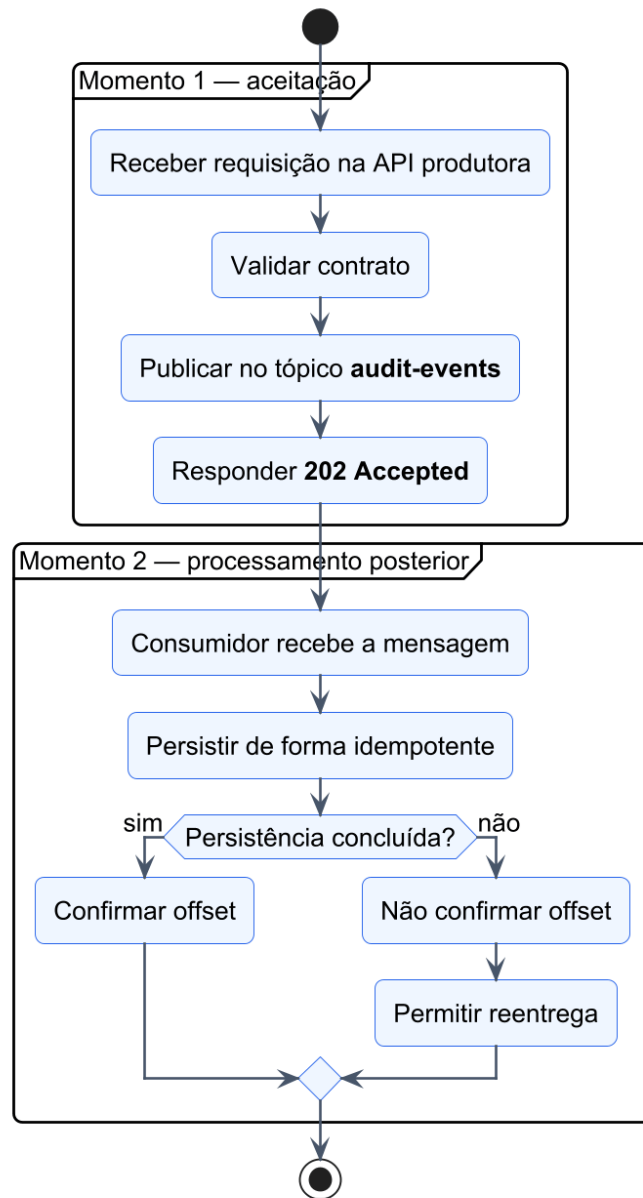


FIGURA 4 – Atividade de processamento assíncrono

FONTE: Elaborado pelo autor (2026).

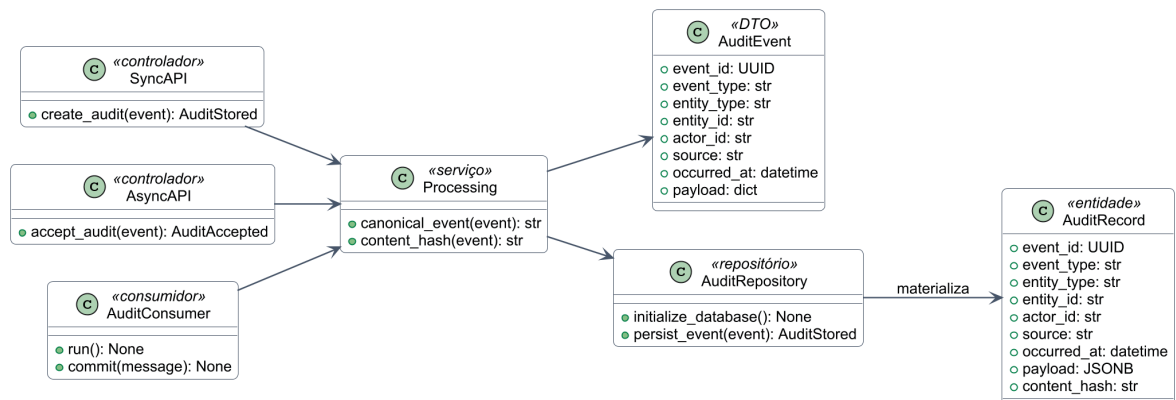


FIGURA 5 – Diagrama preliminar de classes

FONTE: Elaborado pelo autor (2026).

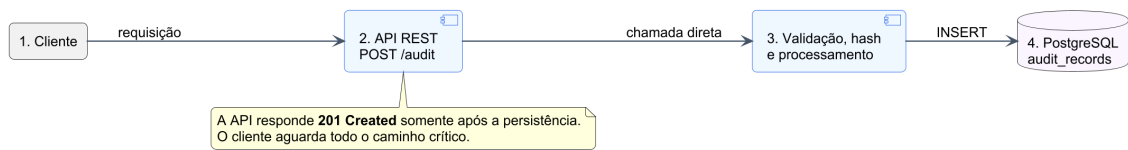


FIGURA 6 – Arquitetura da variante síncrona

FONTE: Elaborado pelo autor (2026).

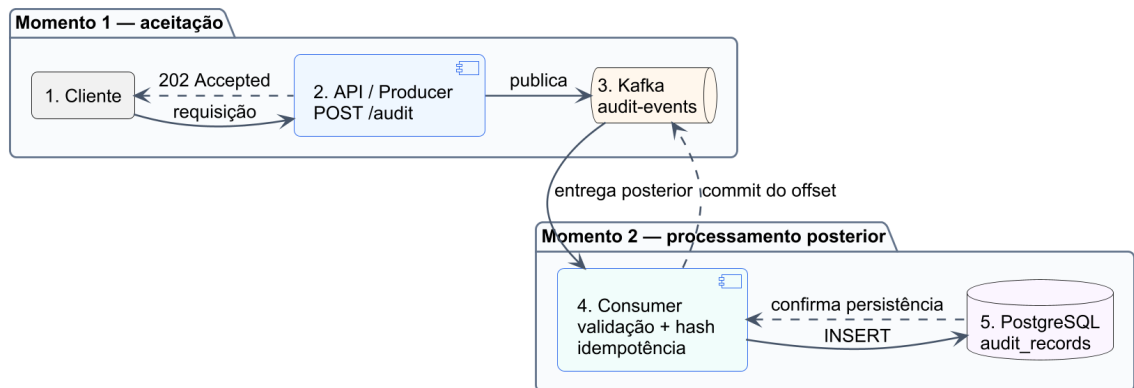


FIGURA 7 – Arquitetura da variante assíncrona

FONTE: Elaborado pelo autor (2026).

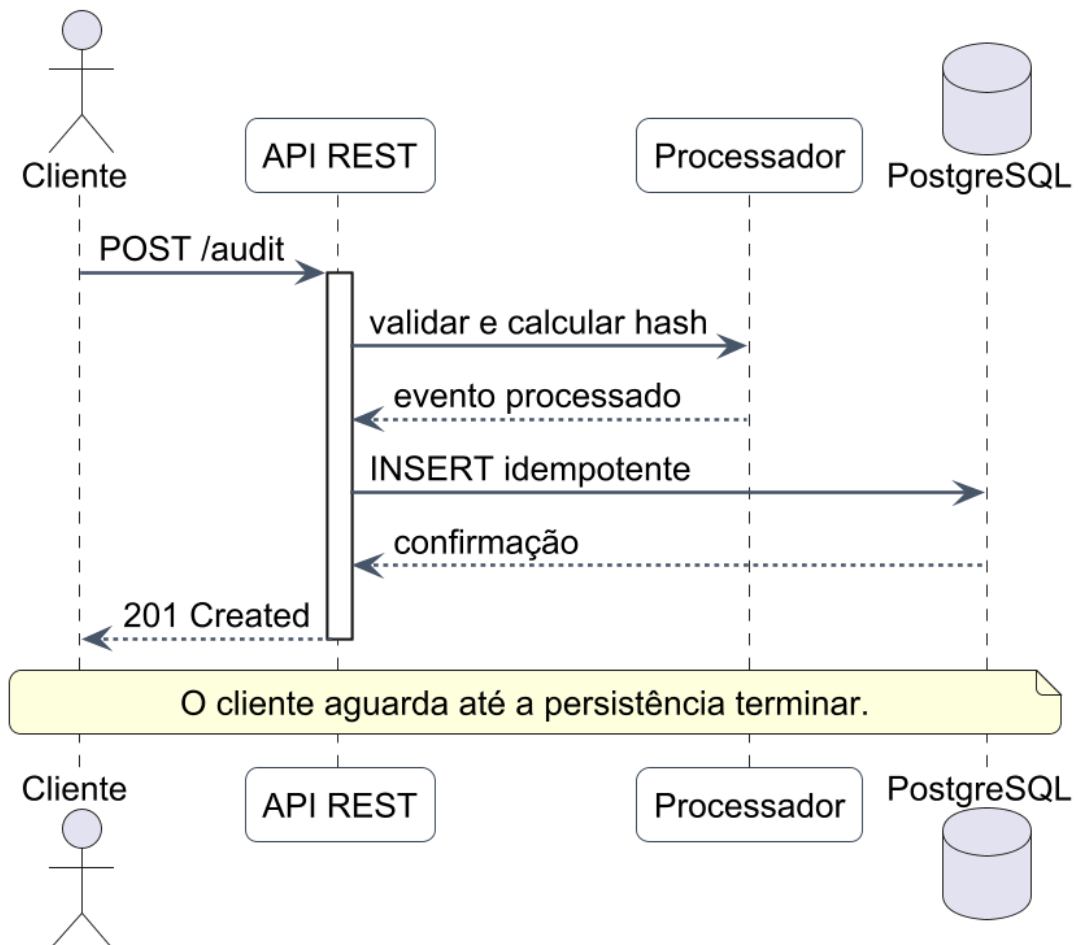


FIGURA 8 – Sequência da variante síncrona

FONTE: Elaborado pelo autor (2026).

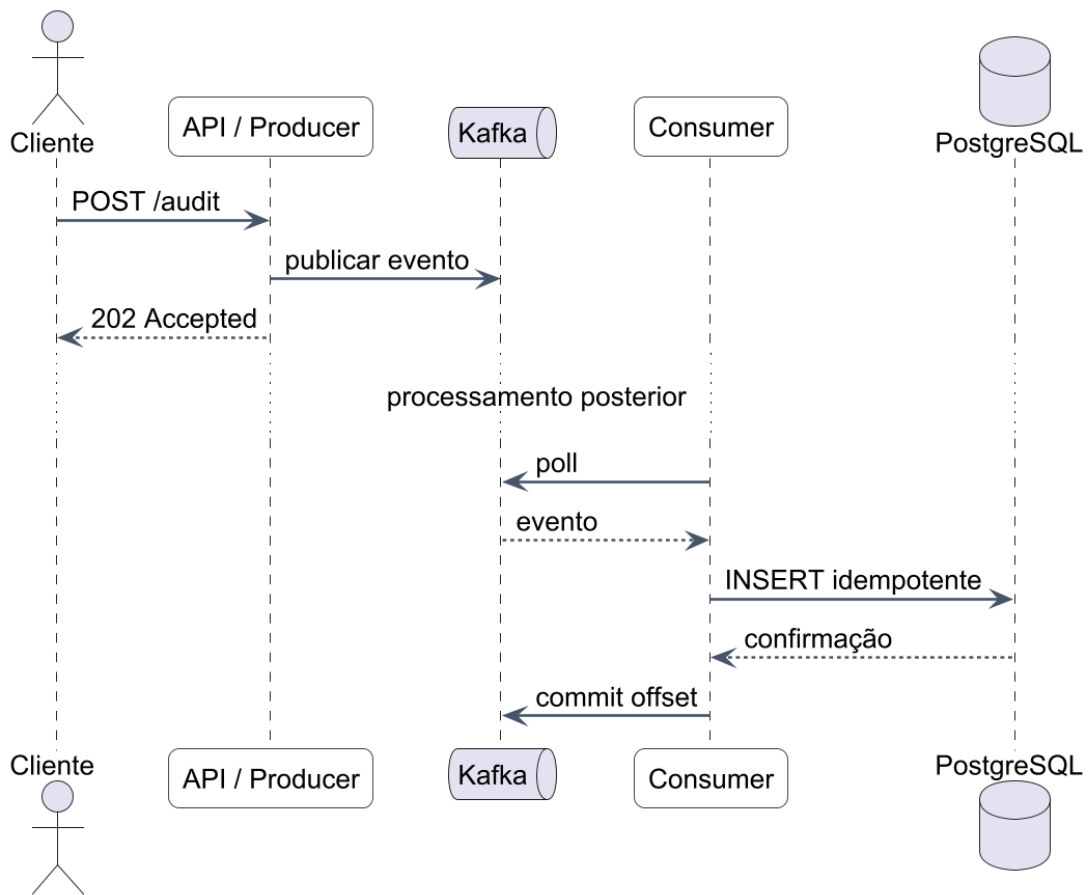


FIGURA 9 – Sequência da variante assíncrona

FONTE: Elaborado pelo autor (2026).

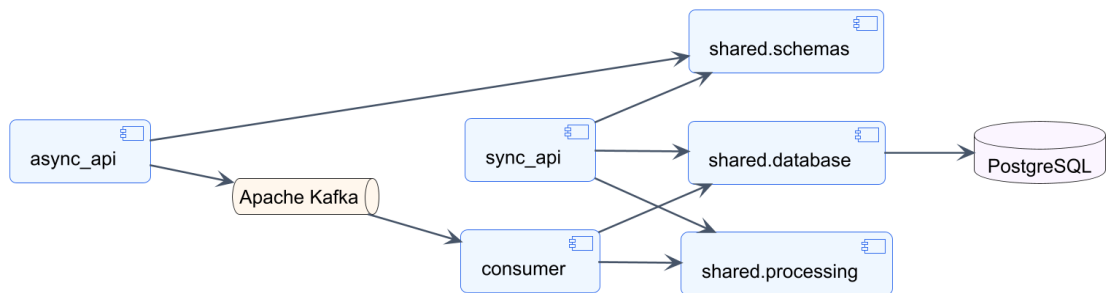


FIGURA 10 – Diagrama de componentes existentes

FONTE: Elaborado pelo autor (2026).

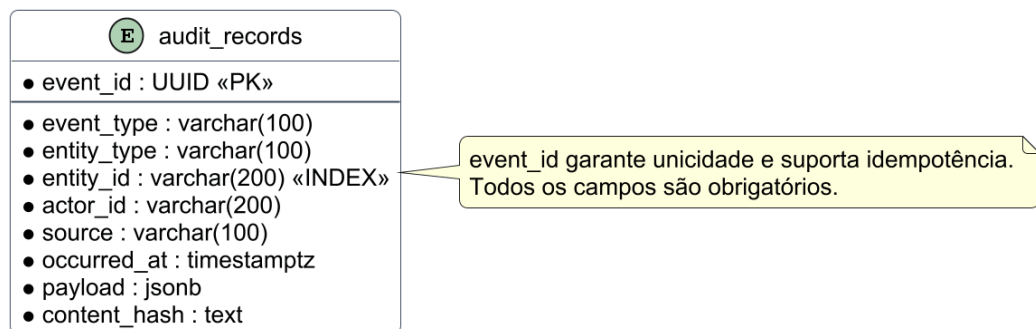


FIGURA 11 – Modelo de dados simplificado

FONTE: Elaborado pelo autor (2026).

REFERÊNCIAS

- AKSAKALLI, I. K. et al. Deployment and communication patterns in microservice architectures: A systematic literature review. **Journal of Systems and Software**, v. 180, p. 111014, 2021.
- Apache Software Foundation. **Apache Kafka: Design**. 2026. Disponível em: <<https://kafka.apache.org/41/design/design/>>. Acesso em: 13 ago. 2026.
- Apache Software Foundation. **Apache Kafka Documentation**. 2026. Disponível em: <<https://kafka.apache.org/documentation/>>. Acesso em: 11 ago. 2026.
- BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYŁEK, A. Monolithic vs. microservice architecture: A performance and scalability evaluation. **IEEE Access**, v. 10, p. 20357–20374, 2022.
- CABANE, H.; FARIAS, K. On the impact of event-driven architecture on performance: An exploratory study. **Future Generation Computer Systems**, v. 153, p. 52–69, 2024.
- FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf>.
- FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <<https://www.rfc-editor.org/rfc/rfc9110.html>>.
- HELLAND, P. Life beyond distributed transactions: An apostate's opinion. In: **Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research**. [s.n.], 2007. p. 132–141. Disponível em: <<https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf>>.
- HOHPE, G.; WOOLF, B. **Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions**. Boston: Addison-Wesley, 2003.
- JAIN, R. **The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling**. New York: Wiley, 1991.
- KAZANAVIČIUS, J.; MAŽEIKA, D. The evaluation of microservice communication while decomposing monoliths. **Computing and Informatics**, v. 42, n. 1, p. 1–36, 2023.
- KENT, K.; SOUPPAYA, M. **Guide to Computer Security Log Management**. [S.l.], 2006. Disponível em: <<https://doi.org/10.6028/NIST.SP.800-92>>.
- LAIGNER, R. et al. Data management in microservices: State of the practice, challenges, and research directions. **Proceedings of the VLDB Endowment**, v. 14, n. 13, p. 3348–3361, 2021.
- OpenTelemetry Authors. **Observability Primer**. 2026. Disponível em: <<https://opentelemetry.io/docs/concepts/observability-primer/>>. Acesso em: 13 ago. 2026.

OWASP Foundation. **Logging Cheat Sheet**. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html>. Acesso em: 11 ago. 2026.

WOHLIN, C. et al. **Experimentation in Software Engineering**. Berlin: Springer, 2012.